

SYSTEM DESIGN

A Complete Prep Guide — Basics to Advanced

For B.Tech CSE Interview Preparation

Covers: Fundamentals • Scaling • Databases • Caching • Load Balancing
CAP/PACELC • Consistent Hashing • Rate Limiting • Bloom Filters
Quorum Consensus • Vector Clocks • WAL & SSTables • 2PC/3PC
Worked Example: Designing a URL Shortener

Table of Contents

1. Part 1 — Foundations: What is System Design, HLD vs LLD
2. Part 2 — Scalability: Vertical vs Horizontal Scaling
3. Part 3 — Networking Basics: Client-Server, DNS, Load Balancers, Proxies
4. Part 4 — Databases: SQL vs NoSQL, Indexing, Replication, Sharding
5. Part 5 — Caching & CDN
6. Part 6 — Asynchronous Systems: Message Queues, Pub-Sub
7. Part 7 — Architecture Styles: Monolith vs Microservices, API Gateway
8. Part 8 — CAP Theorem & PACELC
9. Part 9 — Consistent Hashing
10. Part 10 — Rate Limiting
11. Part 11 — Bloom Filters
12. Part 12 — Quorum Consensus
13. Part 13 — Vector Clocks
14. Part 14 — WAL & SSTables (Storage Engines)
15. Part 15 — 2PC & 3PC (Distributed Transactions)
16. Part 16 — Worked Example: Designing a URL Shortener
17. Part 17 — Interview Framework & Final Checklist

Part 1: Foundations

What is System Design?

System design is the process of defining the architecture, components, modules, and data flow of a software system to satisfy specific requirements — things like how many users it must support, how fast it must respond, and what happens when parts of it fail. In interviews, it tests whether you can take a vague problem ("design Twitter") and turn it into a concrete, justified architecture.

HLD vs LLD

High-Level Design (HLD): the big picture — which services exist, how they talk to each other, what databases are used, where caching/load balancing sits. Think boxes and arrows.

Low-Level Design (LLD): the details inside one box — class diagrams, database schema, API contracts, design patterns used. Think classes and functions.

Example: Designing a food delivery app

HLD: Client app → API Gateway → Order Service, Restaurant Service, Delivery Service → separate databases, with a message queue connecting Order and Delivery services.

LLD: The exact `Order` class, its fields, the state machine for order status (placed → confirmed → out for delivery → delivered), and the REST API schema for `POST /orders`.

Functional vs Non-Functional Requirements

- **Functional requirements** — what the system should do (e.g. "users can post tweets", "users can follow other users").
- **Non-functional requirements** — how well it should do it: scalability, availability, latency, consistency, durability, security.

In an interview, always clarify both before designing. A system built for 1,000 users looks completely different from one built for 100 million users — even if the features are identical.

Back-of-the-envelope estimation

Interviewers expect rough capacity math: requests per second, storage growth per year, bandwidth. You don't need precision — you need to show you can reason about scale.

Example: Estimating storage for a tweet-like app

500 million daily active users, each posting 2 tweets/day, each tweet ~300 bytes of text + metadata.

Daily tweets = 1 billion. Daily storage = $1\text{B} \times 300\text{B} \approx 300\text{ GB/day}$ → roughly 110 TB/year just for text. This tells you you'll need a horizontally sharded database, not a single server.

Part 2: Scalability

Vertical Scaling (Scale Up)

Adding more power (CPU, RAM, disk) to a single existing machine. Simple — no code changes needed — but has a hard ceiling (you can only buy so big a machine) and is a single point of failure.

Horizontal Scaling (Scale Out)

Adding more machines and distributing load across them. Requires a load balancer and often means your application must be stateless, but it scales almost indefinitely and survives individual machine failures.

Vertical Scaling vs Horizontal Scaling

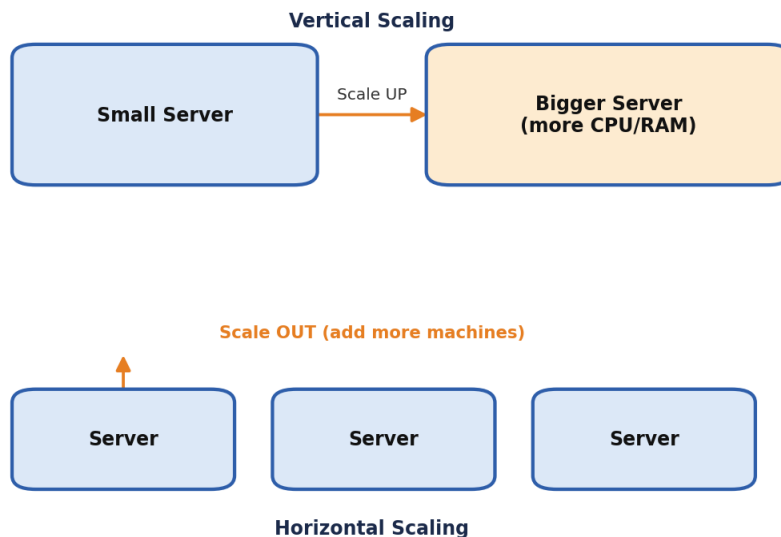


Fig 2.1 — Vertical scaling makes one machine bigger; horizontal scaling adds more machines.

Example: Real-world choice

A startup's internal admin tool with 50 users: vertical scaling is simpler and cheaper. Instagram's photo-serving backend with hundreds of millions of users: must scale horizontally — no single machine could ever handle that load.

Statelessness

A stateless server doesn't store session data locally — it stores it in a shared store (like Redis) or the client (like a JWT token). This is what makes horizontal scaling easy: any request can go to any server because no server "remembers" the user specially.

Part 3: Networking Basics

Client-Server Model

The client (browser, mobile app) sends a request; the server processes it, often talking to a database, and sends back a response. This is the basic unit almost every system design builds on.

Basic Client-Server Communication

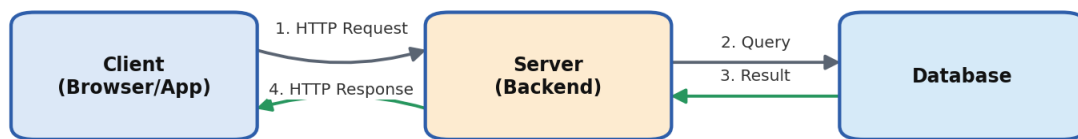


Fig 3.1 — The basic request/response cycle.

DNS (Domain Name System)

Translates a human-readable domain (like *google.com*) into an IP address the network can route to. It's effectively a giant, distributed key-value lookup with heavy caching at every level (browser, OS, ISP).

Load Balancers

Sits in front of multiple servers and distributes incoming requests across them, so no single server is overwhelmed. Common algorithms: Round Robin, Least Connections, IP Hash (same client always goes to the same server — useful for session stickiness).

Load Balancer Distributing Traffic

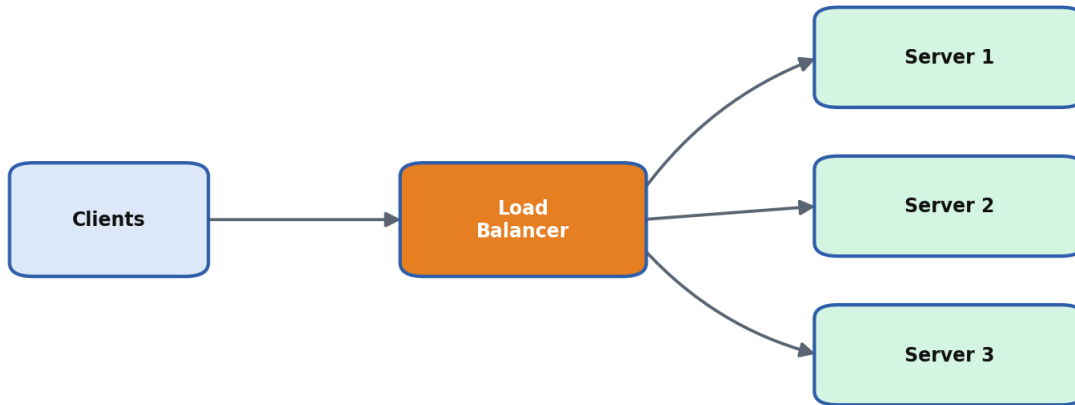


Fig 3.2 — A load balancer spreading requests across three servers.

Example: Why it matters

Without a load balancer, if Server 1 gets 10,000 requests/sec and Server 2 gets 100, Server 1 crashes while Server 2 sits idle. The load balancer prevents this imbalance and also lets you take a server down for maintenance without downtime.

Forward Proxy vs Reverse Proxy

A **forward proxy** sits in front of clients and hides the client's identity from the server (e.g. a company VPN, or bypassing geo-restrictions). A **reverse proxy** sits in front of servers and hides server details from the client — it's what NGINX typically does: routing, SSL termination, compression, and caching, all before a request even reaches your app code.

Forward Proxy vs Reverse Proxy

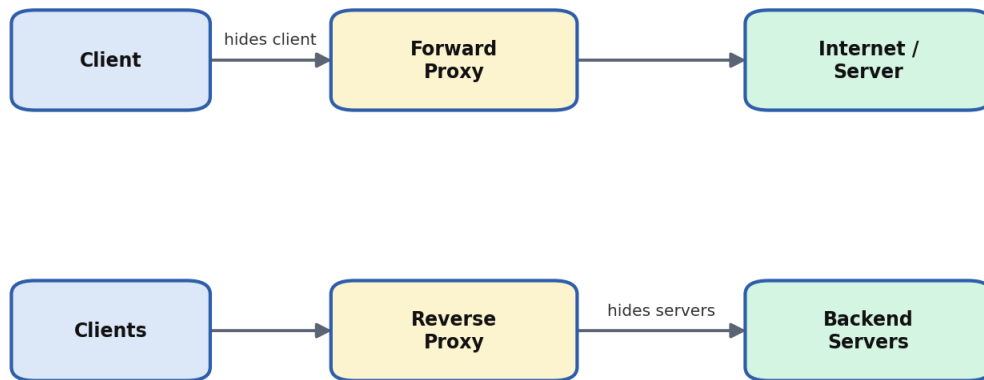


Fig 3.3 — Forward proxy protects the client's identity; reverse proxy protects the servers'.

Part 4: Databases

SQL vs NoSQL

	SQL (Relational)	NoSQL
Schema	Fixed, defined upfront	Flexible / schema-less
Scaling	Vertical (mostly)	Horizontal (built for it)
Consistency	Strong (ACID)	Often eventual (BASE)
Good for	Transactions, relations (banking, orders)	High write throughput, unstructured data (feeds, logs)
Examples	MySQL, PostgreSQL	MongoDB, Cassandra, DynamoDB

Indexing

An index is a data structure (usually a B-Tree) that lets the database find rows without scanning the whole table — like the index at the back of a textbook. It massively speeds up reads (queries filtering or sorting on that column), but slows down writes slightly (the index must be updated too) and takes extra storage.

Example: Indexing in practice

A `users` table with 50 million rows. Query: `SELECT \* FROM users WHERE email = 'x@y.com'`. Without an index, the database scans all 50 million rows. With an index on `email`, it's a near-instant lookup.

Replication

Copying data across multiple database instances for durability and read scalability. In a common leader-follower setup, all writes go to the leader, which then replicates to followers; reads can be served from any follower. If the leader dies, a follower is promoted.

Database Replication (Leader-Follower)

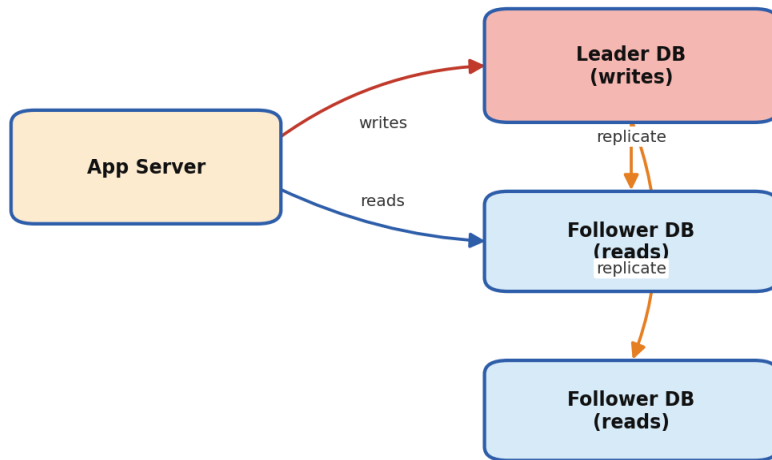


Fig 4.1 — Writes go to the leader; reads can be spread across followers.

Partitioning / Sharding

Splitting one large dataset across multiple database instances (shards), each holding a subset of the data. This is how you scale writes horizontally — a single leader can only take so many writes/sec, but 10 shards can each take that many.

Database Sharding (Horizontal Partitioning)

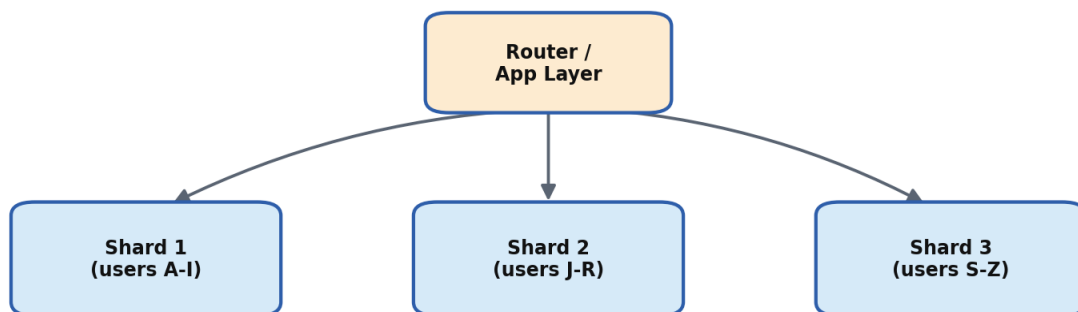


Fig 4.2 — Each shard owns a slice of the data, e.g. by user ID range.

Example: Sharding strategy

An e-commerce database sharded by ``user_id % 4``. User 1042 always lands on shard ``1042 % 4 = 2``. The tricky part: if you later add a 5th shard, almost every key's target shard changes — this is exactly the problem consistent hashing (Part 9) solves.

Part 5: Caching & CDN

Why Cache?

Caching stores frequently-accessed data in fast storage (usually RAM, e.g. Redis or Memcached) so repeated requests don't hit the slower database every time. This cuts latency and reduces database load.

Cache-Aside (Lazy Loading)

The most common pattern: the application checks the cache first. On a hit, it returns immediately. On a miss, it reads from the database, then writes that value into the cache for next time.

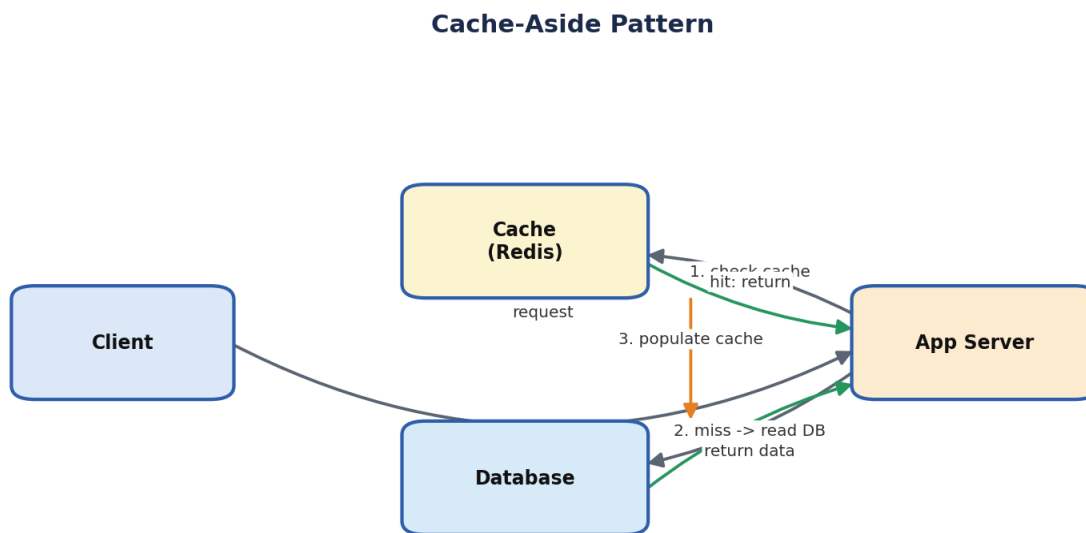


Fig 5.1 — The cache-aside read path: check cache → miss → read DB → populate cache.

Eviction Policies

- **LRU (Least Recently Used)** — evict the item that hasn't been accessed in the longest time. Most common.
- **LFU (Least Frequently Used)** — evict the item accessed the fewest times overall.
- **TTL (Time to Live)** — items expire automatically after a set duration, regardless of usage.

CDN (Content Delivery Network)

A geographically distributed network of edge servers that cache static content (images, videos, JS/CSS) close to the user, so requests don't have to travel all the way to the origin server. This is caching, but at a global, network level rather than an application level.

CDN Serving Static Content

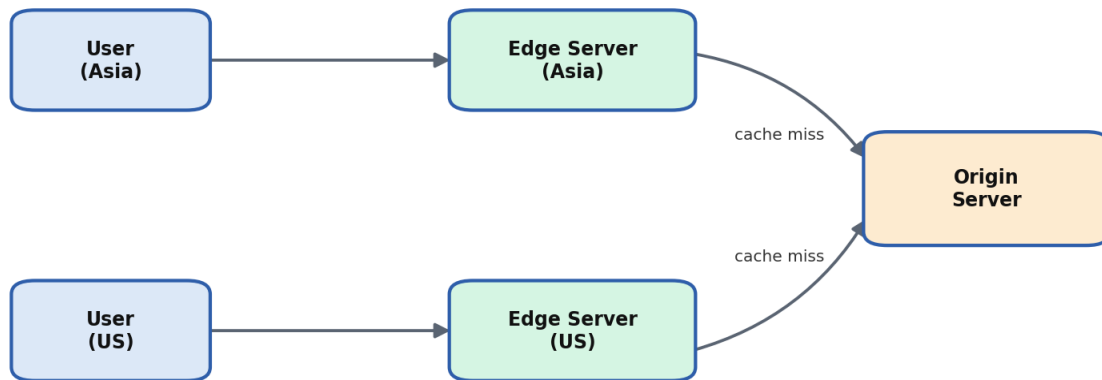


Fig 5.2 — Users hit the nearest edge server; the origin is only contacted on a cache miss.

Example: CDN in action

A user in Mumbai watching a YouTube video is served from a Google edge server in India, not from a data center in the US. This cuts latency from ~250ms to ~20ms.

Part 6: Asynchronous Systems

Message Queues

A queue that decouples the sender (producer) of a message from the receiver (consumer). The producer doesn't wait for the consumer to process the message — it just drops it in the queue and moves on. Common tools: Kafka, RabbitMQ, AWS SQS.

Message Queue (Async Processing)

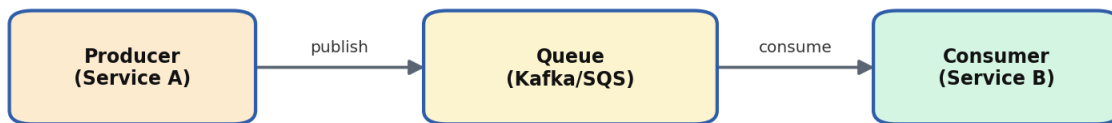


Fig 6.1 — Producer and consumer are decoupled by the queue in between.

Example: Why decoupling matters

When you upload a video to YouTube, the upload service doesn't wait for video transcoding (which can take minutes) before responding to you. It publishes a "new video uploaded" message to a queue, responds immediately, and a separate transcoding service consumes that message whenever it's ready.

Pub-Sub (Publish-Subscribe)

A variant where a message published to a "topic" can be delivered to *multiple* independent consumers, not just one. Useful when several different services all need to react to the same event (e.g. "order placed" triggers both the email service and the analytics service).

Part 7: Architecture Styles

Monolith vs Microservices

A **monolith** is one large codebase/deployable unit containing all functionality. Simple to build and deploy early on, but as it grows, teams start stepping on each other and any small change requires redeploying the whole thing.

Microservices split the system into small, independently deployable services, each owning its own data. This allows teams to work and scale independently, at the cost of added operational complexity — network calls between services, distributed debugging, data consistency across services.

Monolith vs Microservices

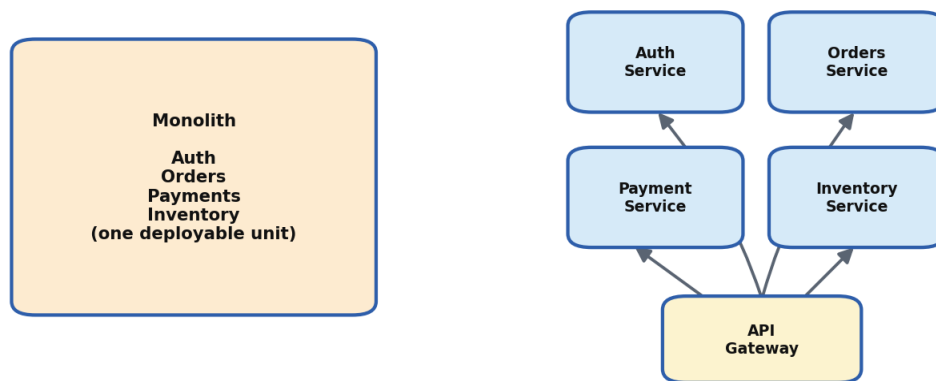


Fig 7.1 — A monolith bundles everything; microservices split by responsibility.

API Gateway

A single entry point that sits in front of a microservices architecture, routing each incoming request to the right backend service. It also commonly handles authentication, rate limiting, and logging in one place, instead of duplicating that logic across every service.

Part 8: CAP Theorem & PACELC

CAP Theorem

In a distributed system, when a network partition happens (nodes can't talk to each other), you must choose between:

- **Consistency (C)** — every read gets the most recent write, or an error.
- **Availability (A)** — every request gets a (non-error) response, even if it might be stale.
- **Partition Tolerance (P)** — the system keeps working despite network failures between nodes.

The theorem says: you can only guarantee 2 of the 3 during a partition. Since real distributed systems *will* experience partitions, P is basically mandatory — so in practice the real choice is **CP vs AP**.

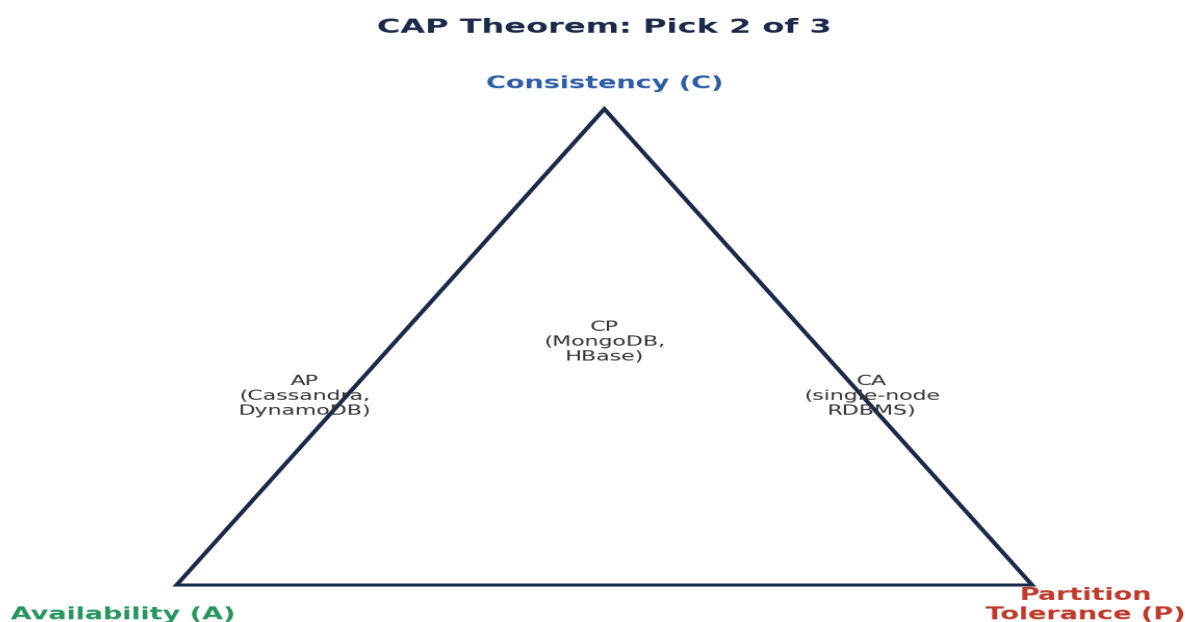


Fig 8.1 — Pick two: a single-node database can be CA, but distributed systems must choose CP or AP.

Example: CP vs AP in practice

A banking system (account balances) favors **CP**: better to show an error than let someone overdraw due to stale data. A social media "like count" favors **AP**: it's fine if the count is briefly out of date, but the app must never go down.

PACELC (an extension of CAP)

CAP only describes behavior *during a partition*. PACELC adds: even when there's **no** partition, you still trade off **Latency (L)** vs **Consistency (C)**, because keeping all replicas perfectly in sync takes time. So: **if Partition, choose A or C; Else, choose L or C.**

Example: PACELC example

DynamoDB is PA/EL: during a partition it favors availability, and normally it favors low latency over strict consistency. MongoDB (with default settings) is more PC/EC: it favors consistency both during partitions and in

normal operation, at some latency cost.

Part 9: Consistent Hashing

The Problem It Solves

With naive hashing ($\text{hash}(\text{key}) \% N$ servers), adding or removing even one server changes the target server for almost every key — causing a massive, unnecessary reshuffle of data across the whole cluster.

How It Works

Both servers (nodes) and keys are hashed onto the same circular range (a "ring"), typically 0 to $2^{32}-1$. A key belongs to the first node found by moving clockwise from the key's position on the ring. When a node is added or removed, only the keys between it and the next node are affected — everything else stays put.

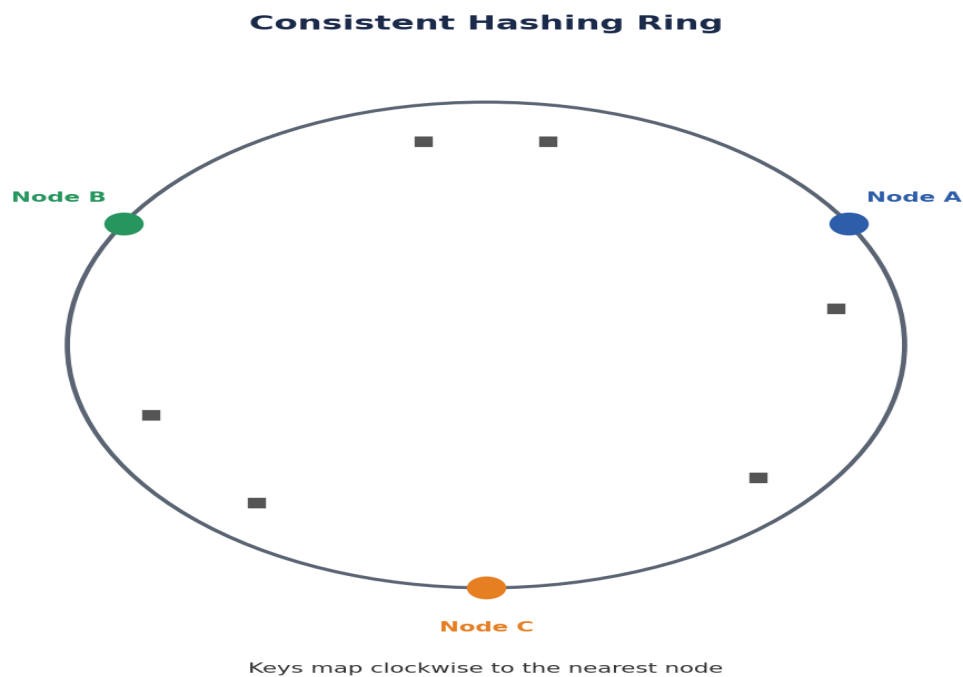


Fig 9.1 — Keys (squares) map clockwise to the nearest node on the ring.

Example: Adding a server

In a ring with Nodes A, B, C, adding Node D only reshuffles the keys between D and the previous node clockwise from it — roughly 1/4 of the data, not everything. Real systems also use "virtual nodes" (each physical server appears at multiple points on the ring) to spread load more evenly.

Part 10: Rate Limiting

Why Rate Limit?

Protects a service from being overwhelmed — by abusive clients, bugs causing retry storms, or genuine traffic spikes — by capping how many requests a client can make in a time window.

Token Bucket Algorithm

A bucket holds tokens, refilled at a fixed rate up to a max capacity. Each request consumes one token. If the bucket is empty, the request is rejected (or queued). This allows short bursts (as long as tokens are saved up) while enforcing a long-term average rate — a good balance most systems use.

Rate Limiting: Token Bucket Algorithm

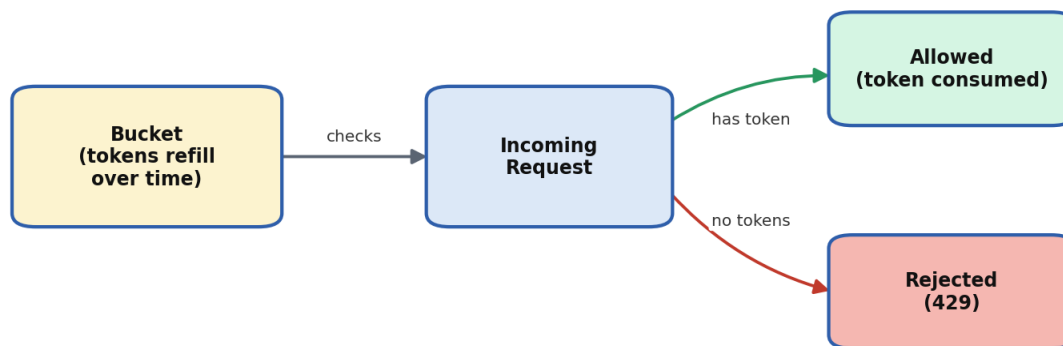


Fig 10.1 — Requests consume tokens; no tokens left means the request is rejected.

Other Algorithms (brief)

- **Leaky Bucket** — requests queue up and are processed at a fixed constant rate, smoothing out bursts entirely.
- **Fixed Window Counter** — count requests in fixed time windows (e.g. per minute); simple but can allow 2x burst at window boundaries.
- **Sliding Window Log/Counter** — tracks requests over a rolling window for smoother, more accurate limiting.

Example: Real-world use

GitHub's API allows 5,000 requests/hour per authenticated user using a token-bucket-like scheme. Exceed it, and you get an HTTP 429 "Too Many Requests" response until your bucket refills.

Part 11: Bloom Filters

What It Is

A space-efficient probabilistic data structure that answers: "is this element possibly in the set?" It can have **false positives** (says "maybe present" when it isn't) but **never false negatives** (if it says "not present", it's definitely not present).

How It Works

A bit array of size m , all initialized to 0. To add an element, run it through k different hash functions, each giving a position in the array, and set those bits to 1. To check membership, hash the element the same way and check if all k bits are set — if any is 0, the element is definitely not present.

Bloom Filter: Fast 'Might Exist' Check

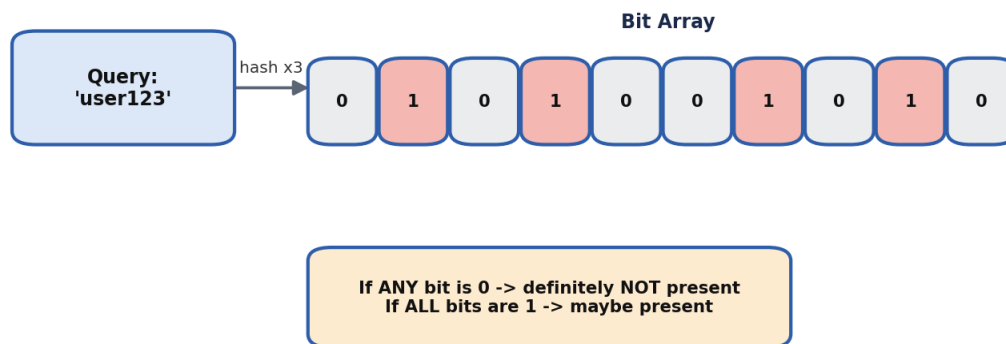


Fig 11.1 — A query hashes to several bit positions; all-1s means "maybe present."

Example: Where it's used

Cassandra uses a bloom filter before checking disk-based SSTables: if the filter says a key definitely isn't there, it skips an expensive disk read entirely. Chrome's Safe Browsing feature uses one to check if a URL is possibly malicious before doing a full (slower) lookup.

Part 12: Quorum Consensus

The Idea

In a system with N replicas of data, instead of requiring all N to acknowledge a write (slow, and fragile if one is down), you require just a **quorum** — a majority subset — for both writes (W) and reads (R).

The Rule: $R + W > N$

If the number of replicas you write to plus the number you read from is greater than the total number of replicas, the read and write sets are guaranteed to overlap by at least one replica — so a read is guaranteed to see the latest write.

Quorum: $R + W > N$ for Consistency

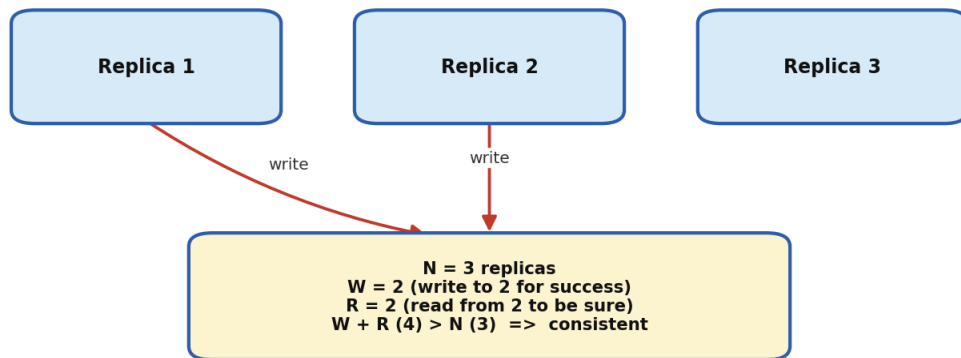


Fig 12.1 — With $N=3$, $W=2$, $R=2$: any read set and write set share at least one node.

Example: Tuning the tradeoff

Cassandra lets you configure W and R per query. Setting $W=1$ makes writes very fast but risks reading stale data; setting $W=N$ (all replicas) makes writes slower but reads can be fast with $R=1$. Most systems pick something balanced, like $W=2$, $R=2$ with $N=3$.

Part 13: Vector Clocks

The Problem

In a distributed system, there's no single global clock — different nodes' clocks drift, so you can't just use timestamps to know which of two conflicting writes happened "first." Vector clocks track **causality** instead: did event A happen-before event B, or are they concurrent (conflicting)?

How It Works

Each node keeps a vector (array) of counters, one per node in the system, e.g. [A:2, B:1, C:0]. Every time a node processes an event, it increments its own counter. When nodes communicate, they merge vectors by taking the max of each position. Comparing two vector clocks tells you if one is a strict descendant of the other (no conflict) or if they diverged independently (a real conflict that needs resolving).

Example: Amazon Dynamo's classic use case

A shopping cart replicated across nodes A and B. You add an item while connected to A (clock becomes [A:1,B:0]); a stale replica sync causes another add via B (clock [A:0,B:1]). Neither vector is an ancestor of the other, so the system detects a genuine conflict and merges both carts, rather than silently discarding one add.

Part 14: WAL & SSTables

Write-Ahead Log (WAL)

Before applying any change to the actual data structure, the change is first appended to a log file on disk. If the server crashes mid-write, it can replay the WAL on restart to recover exactly where it left off. This is how databases guarantee **durability** without needing every single write to be a slow, fully-synced disk operation on the main data structure.

Memtable & SSTables (LSM-Tree storage engines)

Many modern databases (Cassandra, RocksDB, LevelDB) use a Log-Structured Merge Tree: writes go to an in-memory sorted structure (the **memtable**) after being logged in the WAL. When the memtable fills up, it's flushed to disk as an **SSTable** (Sorted String Table) — an immutable, sorted file. Periodically, multiple SSTables are merged ("compaction") to remove duplicates/deletes and keep read performance fast.

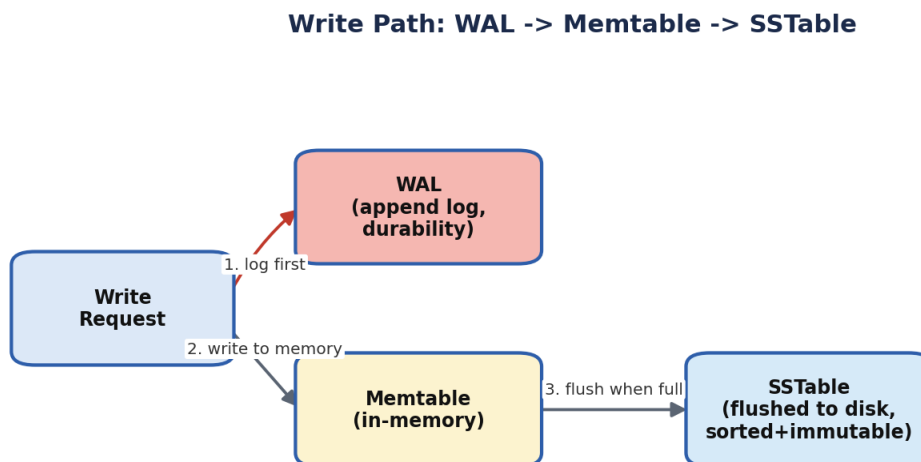


Fig 14.1 — Every write is logged for durability, held in memory, then flushed to disk.

Example: Why this design is fast

Writes are sequential appends (to the WAL and to new SSTables) rather than random-access disk seeks, which is what makes LSM-tree databases like Cassandra extremely fast at write-heavy workloads compared to traditional B-Tree based databases.

Part 15: 2PC & 3PC

The Problem

How do you commit a transaction that spans multiple independent databases/services, ensuring either **all** of them commit or **none** of them do — no partial failures?

Two-Phase Commit (2PC)

- **Phase 1 (Prepare):** a coordinator asks every participant "can you commit?" Each participant locks resources, does the work, and replies YES or NO — but doesn't commit yet.
- **Phase 2 (Commit/Abort):** if all replied YES, the coordinator tells everyone to COMMIT. If even one said NO (or timed out), it tells everyone to ABORT.

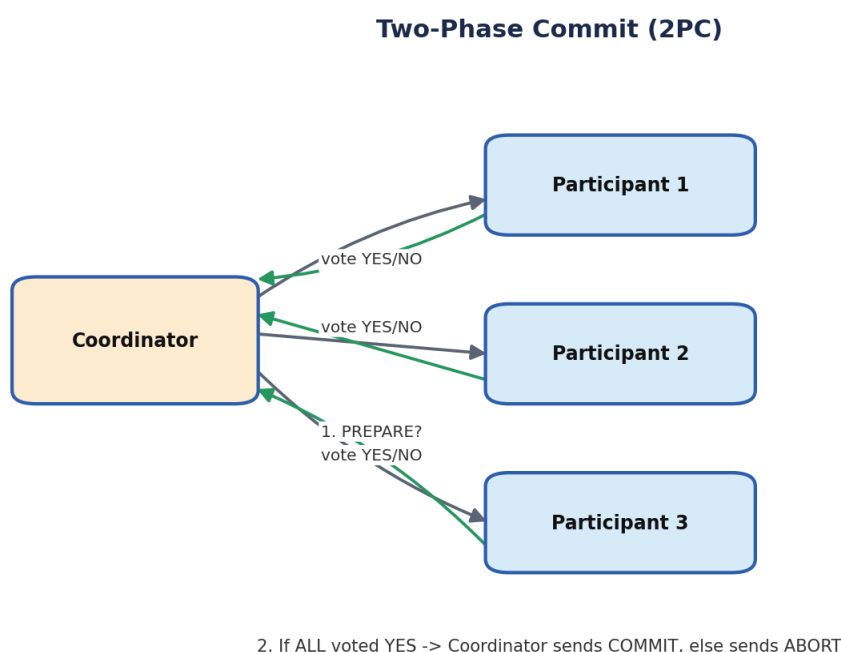


Fig 15.1 — All participants must vote YES before the coordinator commits.

Weakness: if the coordinator crashes after Phase 1 but before sending the final decision, participants are stuck holding locks indefinitely, unsure whether to commit or abort — this is called the "blocking problem."

Three-Phase Commit (3PC)

Adds an extra "pre-commit" phase between prepare and commit, plus timeouts, so participants can make progress independently if the coordinator fails, avoiding indefinite blocking. It's more resilient but adds latency and still doesn't handle network partitions perfectly — which is why many modern systems prefer consensus protocols like **Raft** or **Paxos** instead for critical coordination.

Example: Where 2PC is used

Distributed SQL databases (e.g. Google Spanner, or a payment system debiting one bank's DB and crediting another's) use 2PC-style protocols to guarantee atomicity of a transaction across separate systems.

Part 16: Worked Example — URL Shortener

Let's tie everything together by designing a URL shortener (like bit.ly) end to end — this is one of the most common beginner system design interview questions.

1. Requirements

- Functional: shorten a long URL; redirect a short URL to the original.
- Non-functional: low latency on redirect (<100ms), high availability, ~100M new URLs/month.

2. Capacity Estimate

100M new URLs/month $\approx$ 40 writes/sec on average. Reads (redirects) are typically 100x more frequent than writes $\rightarrow$ ~4,000 reads/sec. This read-heavy ratio tells us caching is critical.

3. Core Design Decisions

- **ID generation:** encode an auto-incrementing counter into base62 (a-z, A-Z, 0-9) to produce short codes like `aZ3kP2`. Avoids collisions without needing to check the database first.
- **Database:** a simple key-value store (`short_code` $\rightarrow$ `long_url`) fits well — NoSQL, sharded by hash of the short code, since there are no complex relations to query.
- **Caching:** cache hot short codes in Redis, since a small fraction of links get the vast majority of traffic (Pareto distribution) — this handles most of that 4,000 reads/sec without touching the DB.
- **Redirect:** use an HTTP 301 (permanent) or 302 (temporary) redirect; 302 lets you keep tracking click analytics on every hit, at a small cost to browser caching.

Example: URL Shortener - High Level Design

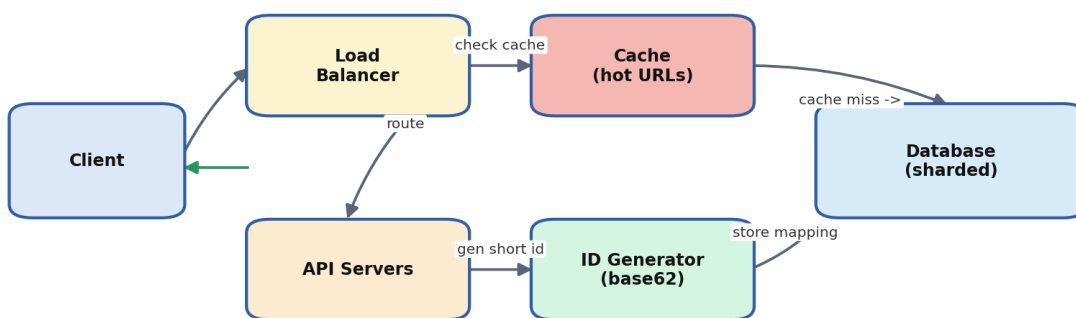


Fig 16.1 — Full request flow: load balancer $\rightarrow$ API servers $\rightarrow$ cache-first read, DB-backed write.

4. Handling Scale

- Load balancer in front of stateless API servers so you can add more servers horizontally as traffic grows.
- Database sharded (Part 4) so writes and storage scale beyond one machine.
- Cache-aside pattern (Part 5) absorbs most read traffic.
- Optionally a CDN in front for extremely hot redirect links.

Example: Interview tip

Always narrate your reasoning out loud: state requirements → estimate scale → propose a simple design → identify the bottleneck → improve it. Interviewers care more about *how* you get to the design than whether you land on the "textbook" answer immediately.

Part 17: Interview Framework & Final Checklist

A Repeatable Framework for Any System Design Question

- **1. Clarify requirements** — functional + non-functional. Ask about scale (users, requests/sec, data size).
- **2. Estimate capacity** — rough numbers for storage, bandwidth, QPS. Shows you understand scale before designing for it.
- **3. Define the API** — what endpoints/interfaces does the system expose? Keeps the design concrete.
- **4. High-level design** — draw the main boxes: client, load balancer, services, database, cache. Keep it simple first.
- **5. Deep dive** — pick 1-2 components the interviewer cares about most and go deep (e.g. "how exactly does sharding work here?").
- **6. Identify bottlenecks & trade-offs** — single points of failure, hot spots, consistency vs availability choices. Propose fixes.
- **7. Wrap up** — summarize the design and mention what you'd improve with more time.

Final Checklist Before Your Interview

- Can you explain CAP theorem with a real example in under 60 seconds?
- Can you draw the read and write path of a cache-aside system from memory?
- Can you explain why consistent hashing is better than `hash % N`?
- Do you know when to reach for SQL vs NoSQL, and can you justify it?
- Can you explain the difference between horizontal and vertical scaling with a concrete example?
- Have you practiced at least 3-4 full designs end-to-end (URL shortener, chat app, news feed, rate limiter)?

Good luck with your interviews. The goal isn't to memorize every term here — it's to understand *why* each piece exists, so you can reconstruct it under pressure and reason about trade-offs on the fly.